

DocAI 系統架構與實作報告

本地端 RAG（檢索增強生成）系統的完整技術分析

版本：1.0 | 日期：2025-12-04



什麼是 DocAI？

DocAI 是一個本地端的 **RAG（檢索增強生成）** 系統。可以把它想像成你的私人圖書管理員，不僅知道每本書存放在哪裡，還能閱讀、理解並回答有關它們的問題。

核心功能

- 上傳並索引 PDF 文件
- 智能理解文件內容
- 快速檢索相關資訊
- 生成精準的答案

01

上傳文件

將 PDF 匯入系統

02

智能索引

AI 閱讀並理解內容

03

快速查詢

提出問題獲得答案

圖書館類比

上傳文件

將書籍（PDF）帶到圖書館

索引建立

圖書管理員閱讀並分類每一頁

智能檢索

快速找到最相關的段落

生成答案

AI 助手閱讀並制定完整答案



雙 RAG 架構設計

DocAI 實作了兩個獨立運作的平行系統,各有其特定用途和優勢。

檔案型 RAG (舊系統)

- 每個檔案獨立索引
- 使用 Milvus 向量儲存
- 5 階段 OPMP 流程
- SSE 串流回應
- 支援多檔案選擇

技能型 RAG (新架構)

- 精選知識領域整合
- 使用 FAISS 向量儲存
- 簡化查詢流程
- JSON 格式回應
- 支援多技能選擇

技術堆疊總覽

	後端框架 Python 3.11 + FastAPI 提供 API 伺服器與非同步處理能力
	向量資料庫 Milvus (檔案) + FAISS (技能) 高效能向量搜尋與輕量級索引
	中繼資料庫 SQLite 儲存技能中繼資料與關聯資訊
	AI 模型 BGE-M3 + Ollama/OpenAI 文字嵌入與回應生成
	前端介面 Vanilla JS + Jinja2 簡潔高效的使用者介面

系統目錄結構

核心程式碼

```
DocAI/  
├── app/  
│   ├── api/v1/endpoints/  
│   ├── models/  
│   ├── Providers/  
│   ├── Services/  
│   └── SkillServices/  
├── template/  
└── scripts/
```

資料儲存

```
data/  
├── faiss_indices/  
│   ├── files/  
│   └── skills/  
├── skill_metadata.db  
└── docai.db
```

api/v1/endpoints

FastAPI 路由處理器,處理 HTTP 請求

Providers

基礎設施層,封裝資料庫與 AI 服務

models

資料模型定義 (TypedDict + Pydantic)

Services

業務邏輯實作,協調各項功能

文件上傳流程

技能型 RAG 系統的完整文件處理流程,從上傳到儲存的每個步驟。



上傳 PDF

使用者透過 API 提交文件



儲存檔案

PDF 存至指定技能目錄



提取文字

PyPDF2 從每頁提取內容



文字分塊

分割成 1000 字元的塊



生成嵌入

BGE-M3 產生向量



雙重儲存

FAISS 存向量,SQLite 存中繼資料

文字分塊策略

分塊參數

1000

塊大小

每個文字塊的字元數

200

重疊長度

保留上下文的重疊字元

分隔符層次

1. 段落分隔 (`\n\n`)
2. 行分隔 (`\n`)
3. 空格分隔
4. 字元分隔

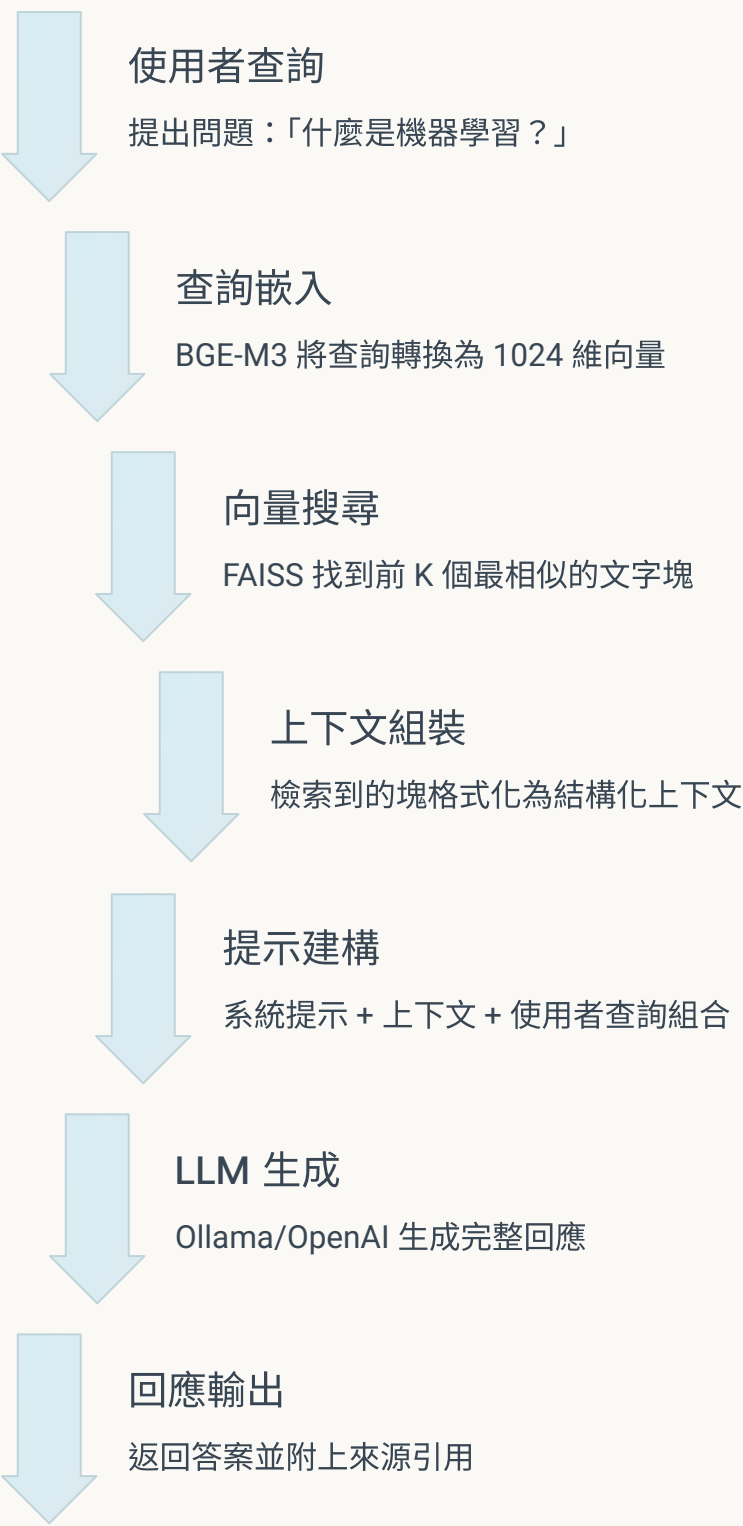
為什麼需要重疊？

重疊設計防止在塊邊界處失去上下文,確保相關文字不會被分割,從而提高檢索品質與答案準確度。

❏ **實際範例：**「機器學習是一個領域。深度學習是子領域。」會被分成有重疊的多個塊,確保「深度學習」的完整上下文被保留。

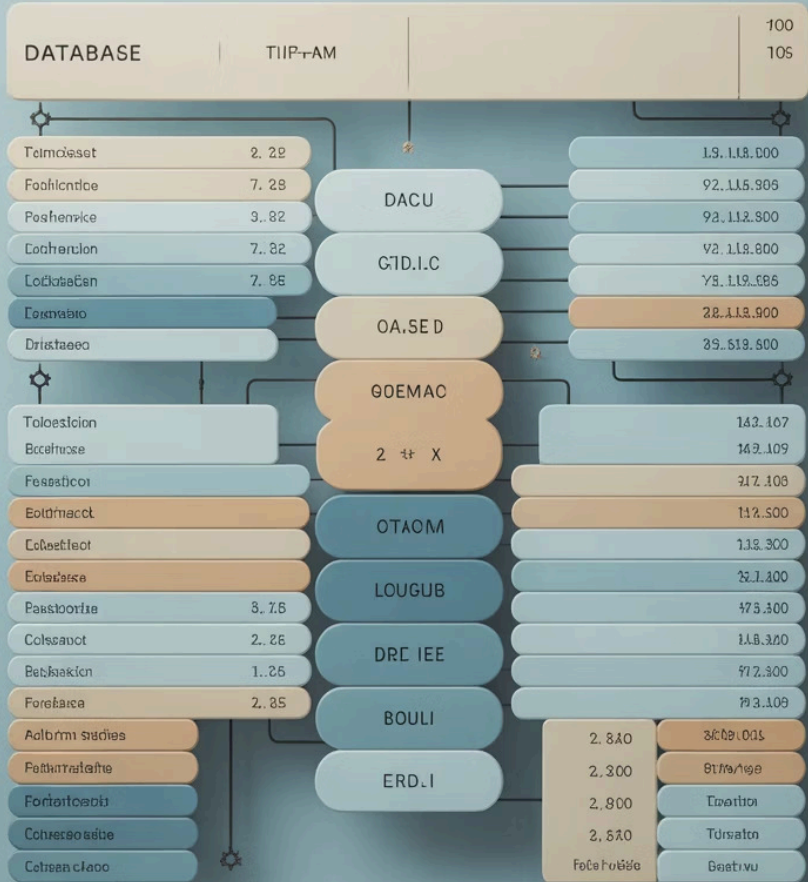
RAG 查詢流程

從使用者提問到獲得答案的完整處理流程。



skill_heads 架構設計

新的 skill_heads 表解決了雙真相來源的反模式,實現單一真相來源的資料管理。



skill_heads 表

技能定義的單一來源

- head_id (主鍵)
- skill_name (唯一)
- description
- category

1:N 關係

一個技能頭可以關聯多個文件記錄

skill_metadata 表

文件記錄詳情

- skill_id (主鍵)
- head_id (外鍵)
- source_name
- total_chunks

架構設計優勢

之前的問題

雙真相來源

skill_config.json + skill_metadata.db 導致同步問題

維護困難

需要手動確保兩個來源的一致性

現在的解決方案



單一真相來源

skill_heads 表統一管理所有技能定義



自動同步

創建新技能自動在 UI 中可見

分層架構總覽

系統採用清晰的分層架構,實現關注點分離與模組化設計。



核心模組：API 端點

app/api/v1/endpoints/skills.py

技能系統的 HTTP 端點處理器,負責所有與技能相關的 API 路由。



POST /skills/create

創建新的技能定義



POST /skills/{name}/upload

上傳 PDF 文件到指定技能



GET /skills/{name}

獲取技能的詳細資訊



DELETE /skills/{name}

刪除指定的技能



GET /skills

列出所有可用的技能



POST /skills/query

在技能中執行查詢

核心模組：文件攝取服務

app/SkillServices/ingestion.py

處理完整的文件攝取流程,從 PDF 提取到向量儲存。

關鍵方法

- `ingest_document()`：主要攝取流程
- `_extract_text()`：PDF 文字提取
- `_chunk_text()`：文字分塊處理
- `_embed_chunks()`：生成向量嵌入
- `_save_to_storage()`：儲存到 FAISS 與 SQLite



塊大小

每個文字塊的字元數



重疊長度

保留上下文的字元數



向量維度

BGE-M3 嵌入維度

核心模組：查詢服務

app/SkillServices/query_service.py

處理 RAG 查詢的核心服務,協調向量搜尋與 LLM 生成。

1

嵌入查詢

將使用者問題轉換為向量

2

向量搜尋

在所有選定技能中搜尋

3

結果重排

按相關性重新排序

4

建構提示

組合上下文與查詢

5

生成回應

LLM 產生最終答案

❏ **多技能搜尋策略：**為每個技能檢索 top_k 個結果,合併後按分數排序,取前 top_k 個整體結果,確保沒有單一技能主導結果。

核心模組：中繼資料服務

app/SkillServices/metadata_service.py

管理技能中繼資料的核心服務,處理所有與 SQLite 資料庫的互動。

主要功能

- 創建技能頭定義
- 添加技能中繼資料記錄
- 檢索技能資訊
- 列出所有技能與來源
- 刪除與更新操作



skill_heads

技能定義表



skill_metadata

文件記錄表



skill_chunks

文字塊儲存表



向量儲存提供者

app/Providers/vector_store.py

向量搜尋的抽象層,使用 FAISS 實作高效能的相似度搜尋。

save_index()

創建 FAISS 索引並儲存至磁碟

similarity_search()

執行向量相似度搜尋,返回前 K 個結果

_load_index()

從磁碟載入索引並快取在記憶體中

為什麼選擇 FAISS ?

無需伺服器

不像 Milvus 需要獨立伺服器,FAISS 是純函式庫

檔案儲存

索引直接儲存為檔案,適合本地部署

快速搜尋

記憶體中的高效能向量搜尋

物理隔離

每個技能擁有獨立的索引目錄



FAISS 索引類型比較

索引類型	速度	準確度	記憶體	適用規模
IndexFlatIP	慢	完美	高	<100K
IndexIVFFlat	中	~99%	中	100K-1M
IndexHNSW	快	~95%	非常高	>1M

DocAI 選擇 IndexFlatIP 的原因：每個技能的向量少於 50K,準確度比速度更重要,且實作簡單無需訓練。

服務類別層次結構

使用策略模式實現可交換的檢索策略,支援檔案型與技能型兩種 RAG 系統。

AbstractRetrievalService

抽象基類定義檢索介面



FileBasedRetrievalService

舊系統：Milvus + OPMP 流程

SkillBasedRetrievalService

新系統：FAISS + 簡化流程

- ❏ **策略模式優勢：**可以在不影響 API 的情況下切換檢索策略,兩個系統共享相同介面,輕鬆進行 A/B 測試。

文字分塊函式詳解

RecursiveCharacterTextSplitter.split_text()

遞迴文字分割器使用分隔符層次結構,確保文字被智能地分割成適當大小的塊。

分隔符層次

- 1. \n\n (段落)
- 2. \n (行)
- 3. 空格 (字)
- 4. 空字串 (字元)

處理流程

01

嘗試分割

使用當前分隔符

02

檢查大小

塊是否超過限制

03

遞迴處理

移至下一個分隔符

04

添加重疊

保留上下文連貫性

嵌入生成函式

EmbeddingProvider.embed_batch()

使用 BGE-M3 模型將文字批次轉換為 1024 維向量嵌入。

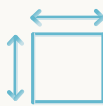


為什麼選擇 BGE-M3 ？



多語言支援

完美支援中文與英文,適合跨語言應用場景



平衡維度

1024 維在儲存大小與檢索品質之間取得最佳平衡



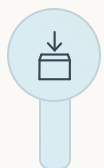
SOTA 效能

在檢索任務上達到業界領先的效能表現

相似度搜尋函式

`VectorStoreProvider.similarity_search()`

在 FAISS 索引中執行高效能的向量相似度搜尋。



載入索引

從快取或磁碟載入 FAISS 索引



正規化查詢

將查詢向量正規化為單位長度



執行搜尋

使用內積計算相似度分數



檢索中繼資料

從 SQLite 獲取塊的文字內容



組合結果

將向量分數與文字內容結合



索引快取優化

為了避免重複載入索引檔案,系統實作了記憶體快取機制。

快取策略

```
self._index_cache: Dict[str, faiss.Index] = {}
```

```
def _load_or_cache_index(skill_id):  
    if skill_id not in cache:  
        path = f"skills/{skill_id}/index.faiss"  
        cache[skill_id] = faiss.read_index(path)  
    return cache[skill_id]
```

10x

速度提升

快取命中時的效能改善

0

磁碟 I/O

快取命中時無需讀取

提示建構函式

PromptBuilder.build_rag_prompt()

為 LLM 建構結構化的 RAG 提示,包含系統指令、上下文與使用者查詢。

提示結構

1 系統提示

定義 AI 助手角色與行為

2 上下文塊

檢索到的相關文字段落

3 使用者查詢

原始問題

4 回答指令

要求包含來源引用

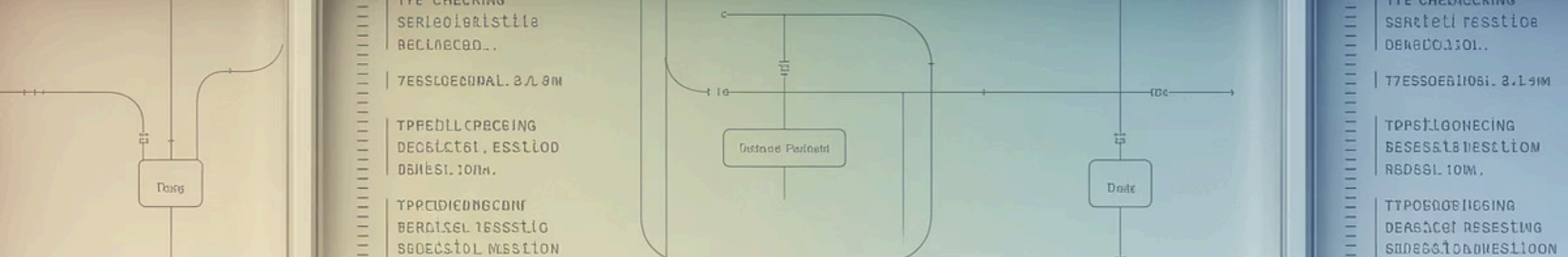
📄 上下文格式

[來源 1: LLM_Book.pdf, 頁 42]

大語言模型是基於 Transformer 架構...

[來源 2: GPT_Guide.pdf, 頁 15]

GPT 使用因果語言建模...



資料模型：TypedDict vs Pydantic

DocAI 使用兩種類型系統的混合方式,各有其特定用途。

TypedDict

用於：內部服務通訊

- 零執行時開銷
- 編譯為常規字典
- IDE 類型提示支援
- 適合高吞吐量場景

Pydantic

用於：API 邊界

- 執行時驗證
- 自動序列化
- OpenAPI 文件生成
- 明確的錯誤訊息

核心 TypedDict 模型



ChunkData

內部塊表示：text_content, chunk_index, page_number, metadata



VectorSearchResult

搜尋結果：content, score, metadata, skill_id, file_id



QueryExpansionResult

查詢增強：original_query, intent, expanded_questions, cache_hit



PromptContext

提示上下文：system_prompt, user_query, context_chunks, chat_history

核心 Pydantic 模型

ChatRequest

```
class ChatRequest(BaseModel):  
    query: str (1-2000 字元)  
    file_ids: List[str] (至少 1 個)  
    session_id: Optional[str]  
    enable_query_enhancement: bool  
    top_k: int (1-20)  
    temperature: float (0.0-2.0)
```

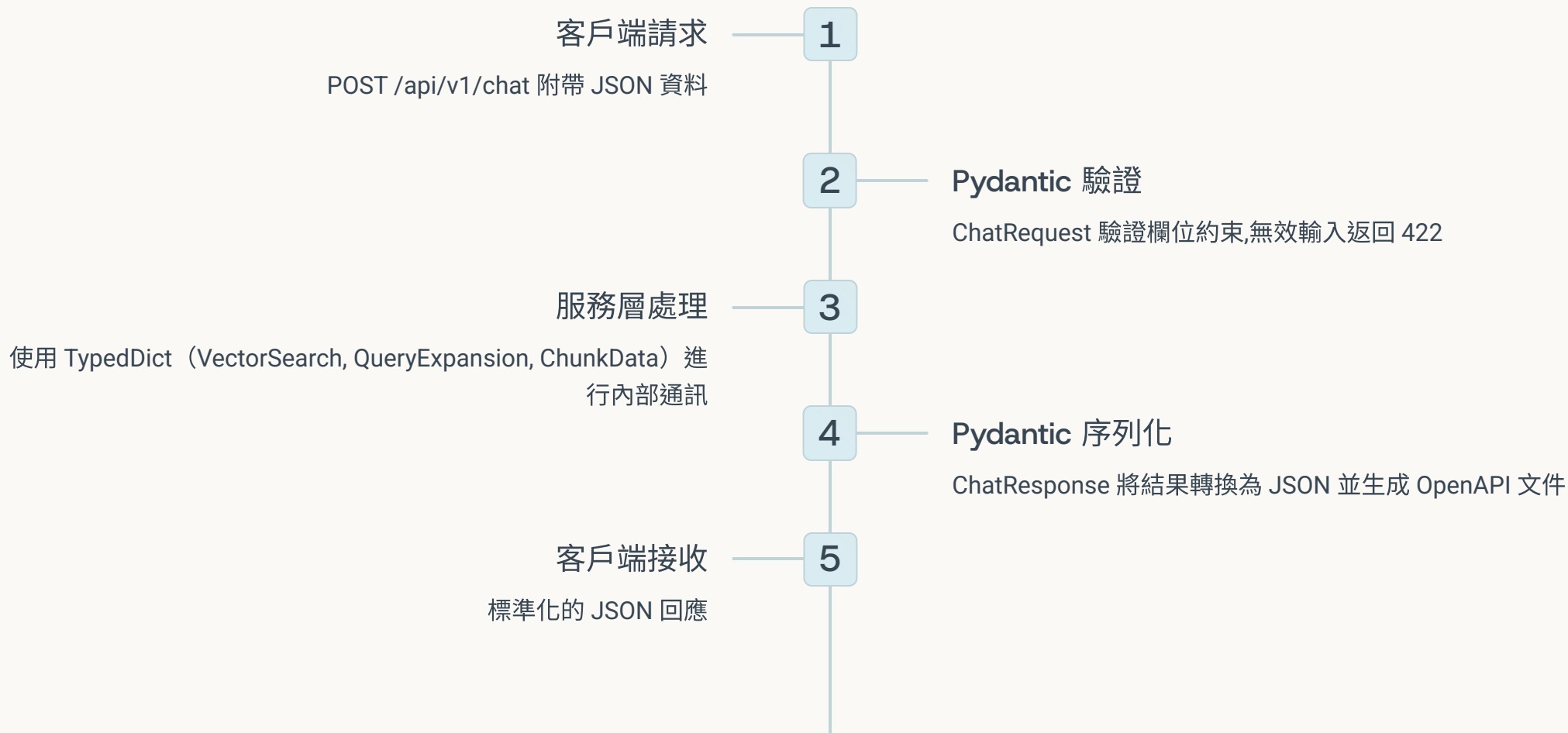
ChatResponse

```
class ChatResponse(BaseModel):  
    answer: str  
    session_id: str  
    query_enhancement: Dict  
    retrieved_chunks: List[Dict]  
    metadata: Dict
```

📄 **驗證優勢：**Pydantic 在 API 邊界自動驗證輸入,無效資料會返回 422 錯誤並附上明確的錯誤訊息。

資料模型使用流程

從 API 請求到內部處理再到回應的完整資料流。



資料庫架構設計

SQLite 三表結構

skill_heads

技能定義表

- head_id (主鍵)
- skill_name (唯一)
- description
- category
- created_at

skill_metadata

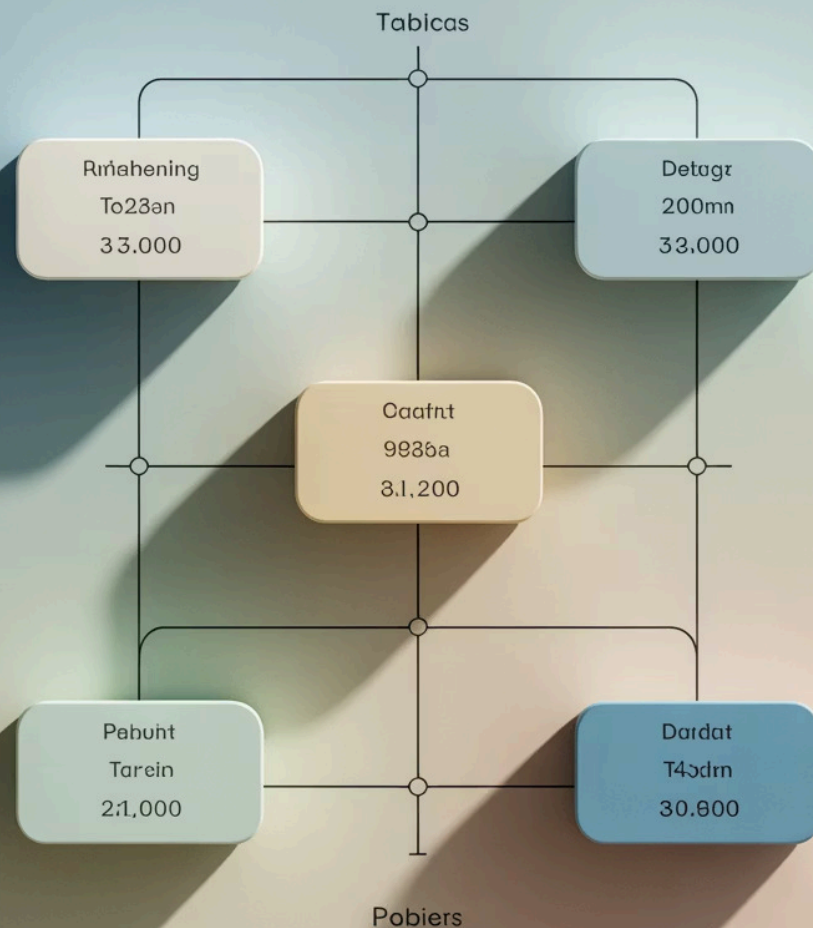
文件記錄表

- skill_id (主鍵)
- head_id (外鍵)
- source_name
- total_chunks
- created_at

skill_chunks

文字塊表

- chunk_id (主鍵)
- skill_id (外鍵)
- chunk_index
- text_content
- page_number
- metadata (JSON)



關鍵設計模式

DocAI 系統中應用的核心設計模式與其帶來的優勢。

儲存庫模式

提供者層（VectorStore、Metadata）封裝資料庫操作, 實現資料存取抽象

策略模式

AbstractRetrievalService 允許可交換的檢索策略, 支援檔案型與技能型切換

工廠模式

get_*_provider() 函式管理單例實例, 確保資源的統一管理

技術選擇理由

技術選擇	理由
FAISS（技能型）	輕量級,無需伺服器,基於檔案的簡單部署
Milvus（檔案型）	高效能向量搜尋,適合大規模檔案管理
SQLite	本地端資料庫,零配置,適合中小型應用
BGE-M3	多語言支援,1024 維平衡,SOTA 檢索效能
TypedDict	內部資料零執行時開銷,保持高效能
Pydantic	API 邊界的自動驗證與文件生成
FastAPI	現代非同步框架,自動 OpenAPI 文件

系統效能特性



向量容量
每個技能支援的最大向量數



嵌入維度
BGE-M3 向量維度



預設 top_k
檢索的文字塊數量



批次大小
嵌入生成的批次處理



錯誤處理機制

攝取流程錯誤處理

```
try:
    # 攝取步驟
except PyPDF2.PdfReadError:
    # 處理損壞的 PDF
except sqlite3.IntegrityError:
    # 處理重複的來源名稱
except Exception as e:
    # 回滾任何部分變更
    # 清理檔案系統工件
    raise IngestionError(f'無法攝取文件：{e}')
```



PDF 錯誤

處理損壞或無法讀取的檔案



資料庫錯誤

處理重複記錄與約束違反



回滾機制

確保資料一致性

系統架構亮點



雙 RAG 系統

檔案型 (Milvus) 和技能型 (FAISS) 獨立運作,各有優勢



分層架構

端點 → 服務 → 提供者 → 儲存,清晰的關注點分離



單一真相來源

skill_heads 表取代 JSON 配置,消除同步問題

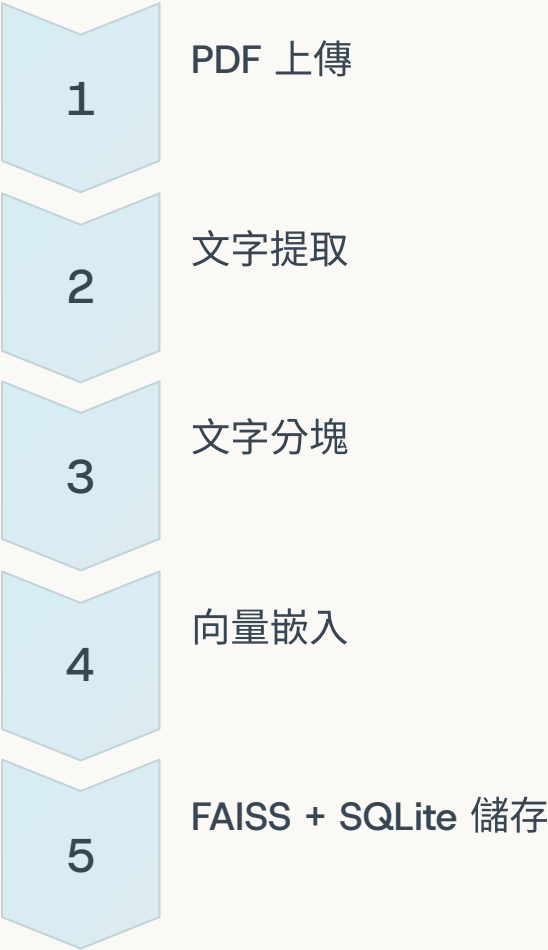


物理隔離

檔案型與技能型索引的獨立目錄結構

完整資料流程總結

上傳流程



查詢流程



未來擴展方向

多模態支援

擴展至圖片、表格等非文字內容的處理能力

分散式部署

支援多節點部署以處理更大規模的資料

即時更新

實現文件的增量更新而非完全重建索引

進階重排

整合更複雜的重排模型提升檢索品質



系統優勢總結

技術優勢

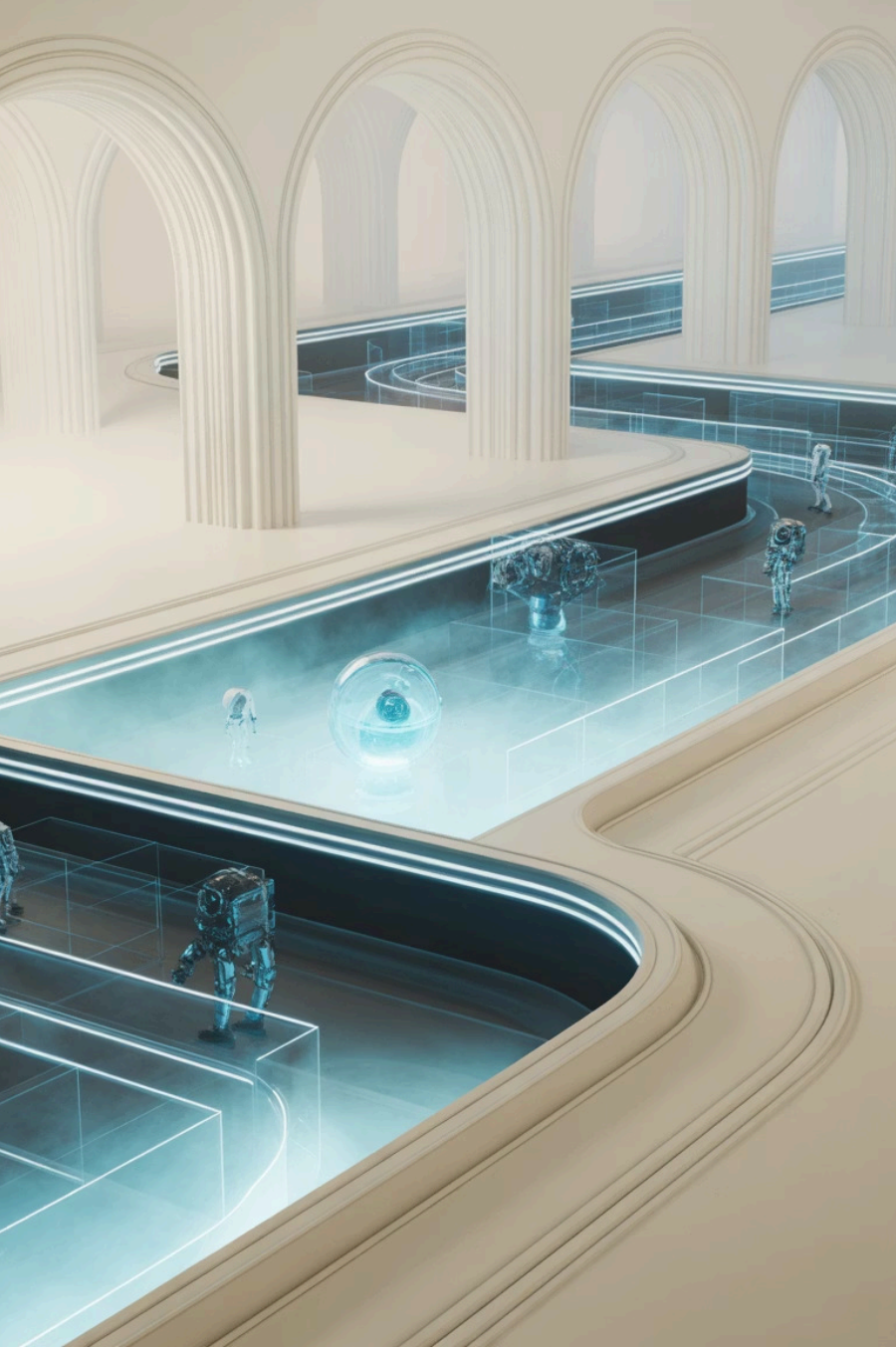
- 本地端部署保護隱私
- 雙 RAG 系統靈活應對
- 高效能向量搜尋
- 模組化設計易於維護

架構優勢

- 清晰的分層結構
- 單一真相來源
- 可交換的策略模式
- 完善的錯誤處理

使用優勢

- 快速文件索引
- 精準的答案生成
- 來源引用追溯
- 多技能整合查詢



總結與展望

DocAI 系統透過精心設計的雙 RAG 架構、分層模組化設計、以及高效能的向量搜尋技術,實現了一個強大且靈活的本地端文件問答系統。

2

RAG 系統

檔案型與技能型並行

4

架構層級

表現、服務、提供者、儲存

3

資料庫表

skill_heads、metadata、
chunks

報告生成時間：2025-12-04 | DocAI 版本：skill_based_docai_dev | 架構：skill_heads 單一真相來源